

背景

本文的受众是通过 Rime 输入引擎来使用字词类输入方案的用户，且这些输入方案涉及到汉字的读音（如音形码、音码）。通常来说，不同于整句类输入方案所使用的 `script_translator`（脚本翻译器），这些方案主要使用 `table_translator`（码表翻译器）。本文将有助于您更好地理解这两个翻译器的性质，并且着重于强调它们的局限性，其中一些可能会解开您关于 Rime 长久以来的疑问或者不满之处。本文的目标在于让更多人意识到这些局限性，并且在更改 libime 主体的情况下，推动新的功能以插件的形式进入各大前端。

本文作者：谭淞宸（蓝落萧），曾基于 Rime 的 lua 接口二次开发实现了声笔输入法 10、冰雪拼音系列输入方案。

问题描述

相比于传统的字词类输入方案平台（例如搜狗五笔、极点、多多等），Rime 的码表翻译器一个明显的优势就是智能算法，包括动态调频（`enable_user_dict`）、组句（`enable_sentence`）、连打造词（`enable_encoder`）、上屏历史造词（`encode_commit_history`）等等。对于纯形码来说，上述功能在使用上基本没有什么问题。但是，对于音形码、形音码和音码来说，则有问题。具体来说，码表翻译器中有两处不同的造词功能：

1. **连打造词**：若一次上屏多个 DictEntry（例如，组句时），则将它们连接的结果造词，造词结果立即进入用户词典；
2. **上屏历史造词**：收集上屏的历史并对其中所有 2 ~ 5 字的组合造词，造词结果先以临时条目的形式进入用户词典，经过再次输入确认之后变为正式词。

然而，这两者都是通过将一段文本反查得到相应的构词码，然后实现造词的。纯形码一般可实现字和构词码的一一对应（容错码除外），但是涉及到汉字读音的时候，反查可能得到多个编码，其中往往只有一个是想要的造词结果。例如，假设「长」有 ca 和 za 两个编码，「度」有 du 一个编码，则用户依次输入「长 ca」「度 du」之后 Rime 会造出 cadu 和 zadu 两个编码，但是用户其实只需要第一个编码。当造词的长度进一步增加的时候，冗余也会进一步增加：例如，如果在一个五字词中有两个字分别有两个读音、一个字有三个读音，则排列组合产生 12 个编码，其中 11 个都是无效的。

为什么脚本翻译器不会产生这个问题？以全拼为例，这是因为全拼的词库中每个字的编码都是它的「全码」，即完整音节；尽管在输入的时候可以采取简拼的形式，但是输入法「知道」这个字的全码，因此在连打造词的过程中可以调用全码来造词。再具体一些：

- 全拼用户用「d」输入「的」，输入法知道它的全码是「de」；
- 小鹤音形用户用「d」输入「的」，输入法不知道它参与二字词时的构词码是「de」，更不知道它的全码是「debu」，除非去反查。

更深一步，这其实是两种翻译器对待编码的两种态度：

- 脚本翻译器认为一个字或词的简码应该由全码通过拼写运算得到
- 码表翻译器认为简码和全码是两个不同的编码（它也支持拼写运算，但大家一般不这么用）

那么，如果用户想用脚本翻译器来实现字词类方案，是否可以呢？部分功能是可以的，但是有些功能不行，例如无法用一二三末的方式来输入多字词。

解决方案

本文计划实现一个新的翻译器，即作为 libime 插件中的一个继承 `Translator` 的类，不妨命名为元翻译器（`MetaTranslator`、`meta_translator`）。它具有以下特点：

1. 基于脚本翻译器实现，向下兼容脚本翻译器的所有功能；
2. 要求方案在码表中以全码标注词的每一个字的编码，这点与脚本翻译器相同；
3. 允许用户指定一些「构词规则」，例如二字词 AaAbBaBb 等等，由输入法推导出词的真正打法；这有点像「音节之间的拼写运算」，但是实际上不是通过正则表达式实现的，而是在音节图（`SyllableGraph`）的构建过程中实现；
4. 像码表翻译器一样同时支持连打造词和上屏历史造词，此时输入法掌握了所有全码信息，因此可以得出唯一合理的造词编码；

5. 用户通过一张额外的表来指定简码以及简码和全码的对应关系，或者用拼写运算来生成简码（这里还需要细化）；
6. 向下兼容码表翻译器的其他功能，例如自动上屏等等。

基本的验证通过之后，本文作者将会推动各大前端内置这一插件，使得以上的功能可以被更多人使用。